Data Science Laboratory Record

Name: Narmadha K **Register No:** 241801181

Contents

• Ex1 - Data Preprocessing and Cleaning (Titanic)

• Ex2 - Data Discovery and Exploratory Data Analysis (Titanic)

• Ex3a - Regression and Clustering (Advertising dataset)

• Ex3b - Regression and Clustering (House Prices dataset)

• Ex4 - Document Retrieval using TF-IDF and Cosine Similarity

• Ex5a - Information Extraction using NLTK

• Ex5b - Information Extraction using spaCy

• Ex6a - Exploratory Data Analysis (Netflix Titles)

• Ex6b - Time Series Analysis and Forecasting using ARIMA

• Ex7 - Model Planning and Building: K-Means Clustering

• Ex8 - Interactive Dashboards with Tableau

Ex1: Data Preprocessing and Cleaning (Titanic)

Aim

To load the Titanic dataset and perform data preprocessing and cleaning: inspect the data, handle missing values (forward fill and backward fill on Age, "Unknown" for Cabin), remove duplicates, encode the categorical column Sex, scale Fare, and visualise relationships using a pair plot and a correlation heatmap.

Code and Explanation

Step 1 - Load the dataset

```
import pandas as pd
df = pd.read_csv("Titanic.csv")
print("Titanic Dataset Loaded Successfully!")
```

pandas is imported and read_csv loads the Titanic file into a DataFrame (891 rows, 12 columns).

Step 2 - Inspect data and missing values

```
print(df.head())
print(df.info())
print(df.isnull().sum())
```

head() shows the first five records, info() shows column types and non-null counts, and isnull().sum() counts missing values per column. Only Embarked had 2 missing values in this file.

Step 3 - Forward fill / backward fill on Age

```
df_forward = df.copy();  df_backward = df.copy()
df_forward["Age"]  = df_forward["Age"].ffill()
df_backward["Age"] = df_backward["Age"].bfill()
df["Age"] = df["Age"].ffill()
```

Copies are made so both methods can be compared. ffill() propagates the last valid value forward and bfill() propagates the next valid value backward. Forward fill is applied to the main dataset.

Step 4 - Fill Cabin with "Unknown"

```
df["Cabin"] = df["Cabin"].fillna("Unknown")
```

Cabin is a mostly empty text column, so missing entries are replaced by a placeholder label.

Step 5 - Remove duplicates

```
print("Duplicate Records:", df.duplicated().sum())
df = df.drop_duplicates()
```

duplicated().sum() found 0 duplicate rows; drop_duplicates() guarantees uniqueness anyway.

Step 6 - Encode Sex and scale Fare

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
df["Sex"] = LabelEncoder().fit_transform(df["Sex"])
df["Fare"] = StandardScaler().fit_transform(df[["Fare"]])
```

LabelEncoder converts the text categories to integers (female = 0, male = 1). StandardScaler transforms Fare to zero mean and unit variance (e.g. 7.25 becomes -0.502445).

Step 7 - Visualisation

```
import seaborn as sns, matplotlib.pyplot as plt
sns.pairplot(df[["Pclass", "Sex", "Age", "SibSp"]]); plt.show()
corr = df[["Pclass","Age","SibSp","Parch","Fare"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm"); plt.show()
```

The pair plot shows distributions and pairwise scatter of selected features. The heatmap shows correlations: Pclass and Fare are strongly negatively correlated (-0.55), while SibSp and Parch are positively correlated (0.41).

Result

The Titanic dataset was cleaned successfully. Missing Age values were filled using forward fill, Cabin was filled with "Unknown", no duplicates were found, Sex was label-encoded, Fare was standardised, and the pair plot and correlation heatmap gave insight into feature relationships.

Ex2: Data Discovery and Exploratory Data Analysis (Titanic)

Aim

To explore the Titanic dataset, impute missing values using SimpleImputer, filter records using conditions to answer analytical questions, and split the data into training and testing sets.

Code and Explanation

Step 1 - Import and load

```
import pandas as pd, seaborn as sns, matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
df = pd.read_csv("titanic.csv")
```

Required libraries are imported and the CSV is converted into a DataFrame.

Step 2 - Understand the data

```
print(df.shape); df.info(); print(df.describe())
```

Shape is (891, 12). Age has 714 non-null values, Cabin 204, Embarked 889. describe() shows mean Age 29.70, mean Fare 32.20 and survival rate 38.4%.

Step 3 to 5 - Handle missing values

```
imputer = SimpleImputer(strategy="mean")
df["Age"] = imputer.fit_transform(df[["Age"]]).ravel()
df["Cabin"] = df["Cabin"].fillna("Unknown")
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

Age is imputed with its mean, Cabin with "Unknown", and Embarked with its most frequent value (mode). After this step all missing counts are zero.

Step 6 - Passenger class countplot

```
sns.countplot(x="Pclass", data=df)
```

Shows the number of passengers per class; class 3 has by far the most passengers (about 490).

Steps 7 to 16 - Conditional filtering

```
df[(df["Sex"]=="female") & (df["Survived"]==1)]           # 7
df[(df["Pclass"]==3) & (df["Age"]<18)]                    # 8
df[(df["Pclass"]==1) & (df["Age"]>40)]                    # 9
df[(df["Pclass"]==1) & (df["Age"]>40) & (df["Survived"]==1)]  # 10
df[(df["Sex"]=="male") & (df["Fare"]>100)]                # 11
df[(df["Embarked"]=="C") & (df["Pclass"]==2)]             # 12
df[df["SibSp"]>2]                                         # 13
df[(df["Survived"]==0)&(df["SibSp"]==0)&(df["Parch"]==0)]  # 14
df[df["Survived"]==1].sort_values("Age", ascending=False).head(5)  # 15
df[df["Fare"]==0]                                         # 16
```

Boolean masks combined with & select rows meeting multiple conditions. Query 15 shows the oldest survivor was Mr. Algernon Henry Wilson Barkworth (80 years). Query 16 lists passengers who paid zero fare.

Step 17 - Train-test split

```
X_numeric = df[["Pclass","Age","SibSp","Parch","Fare"]]
y = df["Survived"]
X_train, X_test, y_train, y_test = train_test_split(
    X_numeric, y, test_size=0.2, random_state=42)
```

Numerical features are separated from the target and split 80/20. random_state=42 makes the split reproducible.

Result

Missing values were handled, 10 analytical queries were answered by filtering, and the data was split into training (712, 5) and testing (179, 5) sets. The final dataset has no missing values.

Ex3a: Regression and Clustering on the Advertising Dataset

Aim

To predict Sales from TV, Radio and Newspaper advertising budgets using Linear Regression, Polynomial Regression and Support Vector Regression, compare them using MAE, MSE and RMSE, and group the data using K-Means clustering.

Code and Explanation

Steps 1 to 3 - Libraries, loading, exploration

```
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("Advertising.csv")
df.info(); print(df.isnull().sum()); display(df.describe())
```

The file is uploaded in Colab and read. It has 200 rows and 5 columns with no null values. Mean Sales is 14.02.

Steps 4 to 6 - Features, pair plot, split

```
X = df[["TV","Radio","Newspaper"]];  y = df["Sales"]
sns.pairplot(df[["TV","Radio","Newspaper","Sales"]])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

The three budgets are the inputs and Sales is the target. The pair plot shows Sales rises strongly with TV and moderately with Radio but weakly with Newspaper. The split gives 160 training and 40 test rows.

Steps 7 to 9 - Train the three models

```
linear_model = LinearRegression().fit(X_train, y_train)
polynomial_model = Pipeline([("poly", PolynomialFeatures(degree=2)),
                             ("linear", LinearRegression())]).fit(X_train, y_train)
svr_model = Pipeline([("scaler", StandardScaler()),
                      ("svr", SVR(kernel="rbf"))]).fit(X_train, y_train)
```

Linear regression fits a straight hyperplane. The polynomial pipeline first adds squared and interaction terms (degree 2) so curved relationships can be captured. SVR with an RBF kernel needs scaled inputs, hence the scaler.

Steps 10 and 11 - Evaluation and comparison

```
def evaluate_model(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    return mae, mse, np.sqrt(mse)
results.set_index("Model")[["MAE","MSE","RMSE"]].plot(kind="bar")
```

MAE is the average absolute error, MSE the average squared error and RMSE its square root (same units as Sales). A grouped bar chart compares the models.

Steps 12 to 14 - K-Means clustering

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(df[["TV","Radio","Newspaper"]])
sns.scatterplot(data=df, x="TV", y="Sales", hue="Cluster", palette="Set1")
```

K-Means groups advertising campaigns into 3 clusters by budget pattern. Cluster sizes were 77, 65 and 58. The scatter plot shows clusters separated mainly along the TV budget axis.

Result

Model comparison:

Model	MAE	MSE	RMSE
Linear Regression	1.4608	3.1741	1.7816
Polynomial Regression	0.5262	0.4129	0.6426
SVR (RBF)	1.1621	2.8621	1.6918

Polynomial Regression gave the lowest error on all three metrics and is the best model for this dataset. K-Means successfully segmented the data into 3 clusters.

Ex3b: Regression and Clustering on the House Prices Dataset

Aim

To predict house price from Area, Bedrooms and house Age using Linear, Polynomial and Support Vector Regression, compare the models using MAE, MSE and RMSE, cluster the houses with K-Means and identify the best model.

Code and Explanation

Steps 1 to 4 - Load and explore

```
df = pd.read_csv("house_prices.csv")
df.info(); print(df.describe()); print(df.isnull().sum())
```

The dataset has 2000 records and 10 columns (Area, Bedrooms, Bathrooms, Floors, YearBuilt, Location, Condition, Garage, Price). There are no missing values. Prices range from 50,005 to 999,656.

Step 5 - Feature engineering

```
current_year = 2023
df["Age"] = current_year - df["YearBuilt"]
X = df[["Area","Bedrooms","Age"]];  y = df["Price"]
```

The original code referenced a non-existent column "Distance", which caused an error. It was corrected by creating Age = 2023 - YearBuilt and using Area, Bedrooms and Age as features.

Steps 6 to 9 - Split and train

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
linear_model.fit(X_train, y_train)
poly_model = Pipeline([("polynomial", PolynomialFeatures(degree=2)), ("linear", LinearRegression())])
svr_model  = Pipeline([("scaler", StandardScaler()), ("svr", SVR(kernel="rbf"))])
```

1600 rows train the models and 400 rows test them. Each model predicts prices for the test set.

Steps 10 to 12 - Evaluation

```
models = {"Linear Regression": y_pred_linear, "Polynomial Regression": y_pred_poly, "SVR": y_pred_svr}
for name, prediction in models.items():
    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    rmse = np.sqrt(mse)
```

The three prediction sets are evaluated in a loop and gathered into a results DataFrame, then plotted with seaborn barplot (melted into long format).

Steps 13 to 15 - Clustering and best model

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X);  df["Cluster"] = kmeans.labels_
sns.scatterplot(data=df, x="Area", y="Price", hue="Cluster")
best_model = results_df.loc[results_df["RMSE"].idxmin()]
```

Houses are grouped into 3 clusters. The clusters split mostly by Area (below about 2000, 2000-3500, above 3500). idxmin() on RMSE selects the best-performing model.

Result

Model comparison:

Model	MAE	MSE	RMSE
Linear Regression	242,644.90	7.7946e10	279,188.70
Polynomial Regression	244,080.06	7.8192e10	279,628.31
SVR (RBF)	242,647.62	7.7903e10	279,110.21

Best model: SVR with the lowest RMSE of 279,110.21. All three errors are very close (and large relative to prices) because the features show almost no relationship with Price in this dataset; predictions stay near the

mean price (about 533,000). Note: in the bar chart the MSE bars dwarf MAE and RMSE because of the scale.

Ex4: Document Retrieval using TF-IDF and Cosine Similarity

Aim

To preprocess Amazon Fine Food Reviews text, convert it into TF-IDF vectors and retrieve the most relevant reviews for a search query using cosine similarity.

Code and Explanation

Steps 1 to 4 - Install, import, NLTK data, download

```
!pip install -q kagglehub nltk scikit-learn pandas
nltk.download('punkt'); nltk.download('punkt_tab'); nltk.download('stopwords')
path = kagglehub.dataset_download("snap/amazon-fine-food-reviews")
```

Libraries are installed, NLTK tokenizer and stopword resources are downloaded, and the dataset is fetched from Kaggle through kagglehub.

Steps 5 and 6 - Load and limit data

```
df = pd.read_csv(csv_file)          # (568454, 10)
df = df[['Text']].copy().dropna().head(10000)
```

The first CSV found is loaded (568,454 reviews). Only the Text column is kept, nulls are dropped and the data is limited to 10,000 reviews for speed.

Steps 7 and 8 - Stopwords and preprocessing

```
stop_words = set(stopwords.words('english'))      # 198 words
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return ' '.join(tokens)
```

Text is lowercased, punctuation and digits are removed, tokenised, stopwords removed, and the tokens joined back into one string.

Steps 9 and 10 - Apply and vectorise

```
df['Cleaned_Text'] = df['Text'].apply(preprocess_text)
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['Cleaned_Text'])   # (10000, 23530)
```

TF-IDF gives each word a weight that is high for words frequent in a review but rare across all reviews. The result is a sparse matrix of 10,000 documents by 23,530 vocabulary words.

Step 11 - Retrieval function

```
def retrieve_documents(query, top_k=5):
    cleaned_query = preprocess_text(query)
    query_vector = vectorizer.transform([cleaned_query])
    scores = cosine_similarity(query_vector, tfidf_matrix).flatten()
    top_indexes = scores.argsort()[-top_k:][::-1]
```

The query goes through the same preprocessing, is converted to a TF-IDF vector, and its cosine similarity with every review is computed. argsort picks the top-k highest scores.

Steps 12 and 13 - Queries

```
retrieve_documents("great product with fast shipping", top_k=5)
retrieve_documents("disappointed", top_k=5)
```

For "great product with fast shipping" the top review scored 0.5024 (enjoyed the product, very fast shipping). For "disappointed" the top review scored 0.485 (a bit disappointed, flavor not as expected).

Result

A TF-IDF based retrieval system was built successfully on 10,000 Amazon food reviews. Relevant reviews were returned in ranked order with similarity scores for both queries.

Ex5a: Information Extraction using NLTK

Aim

To perform information extraction on the Amazon Fine Food Reviews dataset using NLTK: preprocessing, tokenisation, stopword removal, parts-of-speech (POS) tagging and Named Entity Recognition (NER).

Code and Explanation

Steps 1 to 3 - Load, libraries, resources

```
uploaded = files.upload()
df = pd.read_csv('Reviews.csv')
nltk.download('punkt'); nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker_tab'); nltk.download('words')
```

The dataset is uploaded and the NLTK resources needed for tokenising, tagging and chunking are downloaded.

Step 4 - Select reviews

```
df = df[['Text']].dropna(subset=['Text']).head(10000)
```

Keeps only the review text, removes nulls and limits to 10,000 reviews.

Step 5 - Preprocessing

```
def preprocess_text(text):
    text = text.lower()
    return text.translate(str.maketrans('', '', string.punctuation))
```

Converts text to lowercase and removes all punctuation characters.

Step 6 - Tokenisation and stopword removal

```
def tokenize_text(text):
    tokens = word_tokenize(text)
    return [w for w in tokens if w.isalpha() and w not in stop_words]
```

word_tokenize splits the text into words; only alphabetic tokens that are not stopwords are retained.

Step 7 - POS tagging

```
df['POS_Tagging'] = df['Tokens'].apply(pos_tag)
```

pos_tag labels each token with a grammatical tag such as NN (noun), JJ (adjective) or VB (verb).

Step 8 - Named Entity Recognition

```
df['NER'] = df['POS_Tagging'].apply(ne_chunk)
def extract_entities(tree):
    entities = []
    for subtree in tree:
        if hasattr(subtree, 'label'):
            name = ' '.join(w for w, t in subtree.leaves())
            entities.append((name, subtree.label()))
    return entities
```

ne_chunk groups tagged tokens into named-entity subtrees (PERSON, ORGANIZATION, GPE...). The helper walks the tree and returns (entity text, entity label) pairs.

Result

Information extraction was performed successfully on the Amazon Fine Food Reviews dataset using NLTK. The reviews were preprocessed, tokenised and filtered, and POS tagging and Named Entity Recognition were completed.

Ex5b: Information Extraction using spaCy

Aim

To perform information extraction on the Amazon Fine Food Reviews dataset using spaCy for POS tagging and Named Entity Recognition after preprocessing the text.

Code and Explanation

Steps 1 to 3 - Load and set up

```
!pip install pandas spacy -q
import pandas as pd, spacy, string, re
uploaded = files.upload()
df = pd.read_csv('Reviews.csv')
nltk.download('stopwords')
```

Pandas and spaCy are imported, the dataset is uploaded, and NLTK stopwords are used for filtering.

Steps 4 and 5 - Select and preprocess

```
df = df[['Text']].dropna(subset=['Text']).head(10000)
df['Cleaned_Text'] = df['Text'].apply(preprocess_text)   # lowercase + remove punctuation
```

Same selection as Ex5a followed by lowercase conversion and punctuation removal.

Step 6 - Tokenise and remove stopwords

```
def tokenize_text(text):
    tokens = text.split()
    return [w for w in tokens if w.isalpha() and w not in stop_words]
```

Whitespace splitting is used for tokenising; non-alphabetic tokens and stopwords are removed.

Step 7 - POS tagging with spaCy

```
nlp = spacy.load("en_core_web_sm")
def pos_tagging(tokens):
    doc = nlp(" ".join(tokens))
    return [(t.text, t.pos_) for t in doc]
```

The small English model is loaded. Tokens are joined into text, processed by the pipeline, and each token is returned with its universal POS tag (NOUN, ADJ, VERB...).

Step 8 - NER with spaCy

```
def extract_ner(tokens):
    doc = nlp(" ".join(tokens))
    return [(ent.text, ent.label_) for ent in doc.ents]
```

doc.ents holds the entities detected by the model, returned as (text, label) pairs such as (amazon, ORG).

Result

Information extraction was performed successfully using spaCy. The reviews were preprocessed, tokenised and cleaned, and POS tagging and Named Entity Recognition were carried out.

Ex6a: Exploratory Data Analysis on Netflix Titles

Aim

To perform Exploratory Data Analysis on the Netflix Titles dataset and identify patterns in content type, ratings, countries, release and addition trends, durations, genres, people, description text and genre co-occurrence.

Code and Explanation

1 and 2 - Load and explore

```
df = pd.read_csv('netflix_titles.csv')
print(df.shape); df.info(); print(df.describe(include='all'))
print(df.isnull().sum()); print(df.duplicated().sum())
```

Loads the file and checks shape, types, summary statistics, missing values and duplicates.

3 - Content distribution

```
content_counts = df['type'].value_counts()
rating_counts = df['rating'].value_counts()
countries = df['country'].dropna().str.split(', ').explode()
top_countries = countries.value_counts().head(10)
```

Counts Movies vs TV Shows (bar and pie charts), the rating distribution, and the top 10 countries. Multi-country entries are split with str.split and explode so each country is counted separately.

4 - Time trends

```
df['date_added'] = pd.to_datetime(df['date_added'].str.strip(), errors='coerce')
df['year_added'] = df['date_added'].dt.year
growth = df.groupby('year_added').size().sort_index()
```

Release years are counted and plotted as a line chart. date_added is converted to datetime to extract the year added and show Netflix content growth over time.

5 - Duration analysis

```
movies['duration_minutes'] = pd.to_numeric(movies['duration'].str.extract(r'(\d+)')[0])
sns.histplot(movies['duration_minutes'].dropna(), bins=30, kde=True)
tv_shows['seasons'] = pd.to_numeric(tv_shows['duration'].str.extract(r'(\d+)')[0])
```

A regular expression pulls the number from strings like "90 min" or "2 Seasons". A histogram shows movie lengths; a bar chart shows how many TV shows have each number of seasons.

6 - Genres, directors and actors

```
genres = df['listed_in'].dropna().str.split(', ').explode().value_counts().head(10)
directors = df['director'].dropna().str.split(', ').explode().value_counts().head(10)
actors = df['cast'].dropna().str.split(', ').explode().value_counts().head(10)
```

Comma-separated columns are exploded and the top 10 genres, directors and actors are plotted as horizontal bars.

7 and 8 - Word cloud and monthly additions

```
wordcloud = WordCloud(width=1000, height=500, background_color='white',
                      stopwords=WordCloud().stopwords, max_words=100).generate(text)
df['month_added'] = df['date_added'].dt.month
```

A word cloud and a Counter show the most common words in descriptions. The month extracted from date_added shows in which months content is added most.

9 and 10 - Country and rating cross-tabs

```
country_type = pd.crosstab(country_data['country'], country_data['type'])
rating_type = pd.crosstab(df['rating'], df['type'])
```

pd.crosstab counts Movies vs TV Shows per country and per rating, displayed as grouped bar charts.

11 and 12 - Oldest/newest and genre co-occurrence

```
oldest_title = df.loc[df['release_year'].idxmin()]
newest_title = df.loc[df['release_year'].idxmax()]
for pair in combinations(sorted(set(genre_list)), 2):
    genre_pairs[pair] += 1
sns.heatmap(cooccurrence, annot=True, fmt='g', cmap='YlGnBu')
```

idxmin/idxmax find the oldest and newest release. itertools.combinations counts how often two genres appear together in the same title, and the matrix is shown as a heatmap.

Result

EDA on the Netflix Titles dataset was completed successfully. Content distribution, ratings, country-wise content, release and addition trends, movie durations, TV show seasons, popular genres, directors, actors, description keywords, monthly additions and genre co-occurrence patterns were analysed using statistics and visualisations.

Ex6b: Time Series Analysis and Forecasting using ARIMA

Aim

To analyse monthly airline passenger data, check stationarity, apply differencing, build an ARIMA model, forecast passenger numbers and evaluate the forecast accuracy.

Code and Explanation

Steps 1 to 3 - Load with DatetimeIndex

```
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url, parse_dates=['Month'], index_col='Month')
df.index = pd.DatetimeIndex(df.index)
```

The Airline Passengers file is downloaded from GitHub. parse_dates and index_col make Month a time index, which time-series models require.

Step 4 - Plot and decompose

```
plt.plot(df['Passengers'])
decomposition = seasonal_decompose(df['Passengers'], model='multiplicative', period=12)
decomposition.plot()
```

The plot shows an upward trend with yearly seasonality that grows over time. Multiplicative decomposition separates trend, seasonal and residual parts (period = 12 months).

Step 5 - ADF stationarity test

```
result = adfuller(df['Passengers'])
print("ADF Statistic:", result[0]); print("p-value:", result[1])
```

The Augmented Dickey-Fuller test checks for a unit root. If p-value is 0.05 or less the series is stationary; otherwise it is non-stationary and needs differencing.

Steps 6 to 8 - Differencing

```
df['First_Difference'] = df['Passengers'].diff()
differenced_series = df['First_Difference'].dropna()
result_diff = adfuller(differenced_series)
# if still non-stationary: log transform, then second difference (d = 2)
```

Differencing subtracts each value from the previous one to remove the trend. ADF is re-run on the differenced series, falling back to log transform or second-order differencing, and the selected order d is stored.

Step 9 - Train/test split

```
train_size = int(len(series) * 0.8)
train = series.iloc[:train_size];  test = series.iloc[train_size:]
```

Time series are split chronologically (no shuffling): the first 80% (115 months) train, the last 20% (29 months) test.

Steps 10 and 11 - ARIMA and forecast

```
model = ARIMA(train, order=(2, d, 2))
model_fit = model.fit()
forecast = model_fit.forecast(steps=len(test))
```

ARIMA(p, d, q) uses p autoregressive terms, d differences and q moving-average terms. The model is fitted on the training data and forecasts as many steps as the test set.

Steps 12 and 13 - Metrics and plot

```
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)
plt.plot(train); plt.plot(test); plt.plot(forecast, linestyle='--')
```

MSE, MAE and RMSE measure forecast accuracy against the actual test values. One plot shows training data, actual test data and the ARIMA forecast together.

Result

Time series analysis and forecasting were performed successfully on the Airline Passengers dataset. Stationarity was checked with the ADF test, differencing was applied, an ARIMA model was fitted, the test period was forecast and accuracy was evaluated using MSE, MAE and RMSE.

Ex7: Model Planning and Building - K-Means Clustering

Aim

To segment mall customers by Annual Income and Spending Score using K-Means clustering, find the optimal number of clusters with the Elbow method, visualise the clusters and recommend marketing strategies.

Code and Explanation

Steps 1 and 2 - Load and explore

```
df = pd.read_csv('Mall_Customers.csv')
print(df.shape, df.dtypes, df.isnull().sum(), df.duplicated().sum())
```

Loads the Mall Customers dataset and checks shape, types, missing values, duplicates and summary statistics.

Steps 3 and 4 - Feature selection and scaling

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

The two clustering features are selected and standardised so that both contribute equally to the distance calculations used by K-Means.

Step 5 - Visualise features

```
plt.scatter(X['Annual Income (k$)'], X['Spending Score (1-100)'])
sns.pairplot(df[['Annual Income (k$)', 'Spending Score (1-100)']])
```

A scatter plot and pair plot show the natural groupings in the data before clustering.

Step 6 - Elbow method

```
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    km.fit(X_scaled);  wcss.append(km.inertia_)
```

WCSS (within-cluster sum of squares) is computed for K = 1 to 10. The point where the curve bends (the elbow) indicates the best K, which is 5 for this dataset.

Step 7 - Train and save

```
kmeans = KMeans(n_clusters=5, init='k-means++', random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)
df.to_csv('Mall_Customers_Clustered.csv', index=False)
```

The final model with K = 5 assigns each customer a cluster label, stored in a new column and saved to a CSV file.

Step 8 - Cluster visualisation with centroids

```
centroids_original = scaler.inverse_transform(kmeans.cluster_centers_)
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)',
                hue='Cluster', palette='tab10', s=80)
plt.scatter(centroids_df.iloc[:,0], centroids_df.iloc[:,1], marker='X', s=250, color='black')
```

Centroids are in scaled units, so inverse_transform converts them back to real values. They are drawn as black X markers with their coordinates annotated.

Steps 9 and 10 - Cluster analysis and strategies

```
cluster_analysis = df.groupby('Cluster').agg(
    Customer_Count=('CustomerID', 'count'),
    Average_Annual_Income=('Annual Income (k$)', 'mean'),
    Average_Spending_Score=('Spending Score (1-100)', 'mean')).round(2)
```

Customers per cluster and average income and spending score are computed. If-else rules then print a strategy per cluster: high income and high spending get loyalty rewards; high income and low spending get targeted discounts; low income and high spending get affordable premium deals; low income and low spending get budget-friendly promotions; the rest get personalised offers.

Result

Customer segmentation was performed successfully using K-Means. Features were standardised, the Elbow method gave K = 5, customers were grouped into clusters, the centroids were visualised, cluster sizes and averages were computed, and suitable marketing strategies were recommended for each segment.

Ex8: Interactive Dashboards with Tableau

Aim

To create an interactive dashboard in Tableau using the Sample Superstore dataset, with a bar chart of Sales by Category, a line chart of Sales trend over time, a scatter plot of Profit vs Sales by Sub-Category, and interactive filters and dashboard actions.

Code and Explanation

1 and 2 - Load and explore data

Open Tableau, connect to the Superstore file (Excel or text), and check the Data pane. Order Date must be a Date field; Sales, Profit and Quantity must be numeric measures; Category, Sub-Category and Region must be present.

3 - Bar chart: Sales by Category

Drag Category to Columns and SUM(Sales) to Rows, sort descending, drag Region to Color and show mark labels. This shows total sales for Furniture, Office Supplies and Technology split by region.

4 - Line chart: Sales trend

Drag Order Date to Columns as Month (continuous), SUM(Sales) to Rows and Region to Color. Separate lines reveal growth, declines and seasonal patterns.

5 - Scatter plot: Profit vs Sales

Put SUM(Sales) on Columns and SUM(Profit) on Rows, add Sub-Category to Detail and Category to Color. Points below zero profit are loss-making sub-categories (for example Tables and Bookcases).

6 - Build the dashboard

Create a new dashboard (about 1200 x 800), place the line chart on top, the bar chart bottom-left and scatter plot bottom-right using layout containers, and add the title "Sales and Profit Dashboard".

7 - Interactivity

Show filters for Region, Category and Order Date and apply them to the relevant worksheets. Add a Filter action (source: bar chart, targets: line chart and scatter plot, run on Select) and an optional Highlight action on Hover so selections update the other charts.

8 - Publish or export

Publish to Tableau Public or export the dashboard as PDF/image, then verify filters, actions and title.

Result

An interactive dashboard was created in Tableau using the Sample Superstore dataset. The bar chart, line chart and scatter plot analyse category-wise sales, sales trends over time and the sales-profit relationship. Interactive filters and actions enable dynamic exploration, and the dashboard was published or exported for reporting. (Dashboard screenshots are in your original record.)